

## Code Review - Calculadora Pytest

|         |                                                                       |
|---------|-----------------------------------------------------------------------|
| Trilha  | AWS & AI for Software Quality Engineering                             |
| Período | Nov/2025                                                              |
| Fase    | 02                                                                    |
| Sprint  | 01                                                                    |
| Semana  | 02                                                                    |
| Squad   | 01                                                                    |
| Membros | Ana Leticia De Araujo; Eduardo Neves de Souza; Gustavo Santos Queiroz |

### Code Reviews

01. Autor do código: Ana Leticia De Araujo

Autor do Review: Eduardo Neves de Souza

Link do repositório: [PB-Compass/Documents/Fase 2/Semana 2 at main · analeticiaraujo/PB-Compass](#)

Resumo do review:

 A calculadora realiza corretamente as operações básicas e os testes com pytest cobrem diferentes cenários, incluindo casos de erro como divisão por zero. O uso de parametrize foi uma boa escolha, pois evita repetição de código nos testes.

Alguns pontos podem ser melhorados. Em várias funções, a variável valorfinal é usada desnecessariamente, pois o resultado poderia ser retornado diretamente. Além disso, a função que calcula o resto da divisão poderia usar o operador %, que faz esse cálculo de forma mais simples e segura e garante uma maior cobertura, como em restos da divisão de números negativos que atualmente acabam por exibir o resultado errado.

De maneira geral, o código funciona e os testes validam adequadamente as operações, mas ainda há espaço para simplificar algumas partes da implementação.

Código feito pelo autor:  
calculadora.py

```
1  ### CALCULADORA BASICA ###
2  import pytest
3
4  valorfinal = float
5
6  class Calculadora():
7      def soma(self, numero1, numero2):
8          return numero1 + numero2
9      def subtracao(self, numero1, numero2):
10         return numero1 - numero2
11     def multiplicacao(self, numero1, numero2):
12         return numero1 * numero2
13     def divisao(self, numero1, numero2):
14         if numero2 == 0:
15             raise ZeroDivisionError("Não é possível dividir por 0")
16         else:
17             return numero1 / numero2
18     def exponencial(self, numero1, numero2):
19         return numero1 ** numero2
20     def divisaoresto(self, numero1, numero2):
21         if numero2 == 0:
22             raise ZeroDivisionError("Não é possível dividir por 0")
23         else:
24             return numero1 - (int(numero1 / numero2) * numero2)
```

```

25
26
27 if __name__ == "__main__":
28     calculadora = Calculadora()
29     operador = {
30         1: calculadora.soma,
31         2: calculadora.subtracao,
32         3: calculadora.multiplicacao,
33         4: calculadora.divisao,
34         5: calculadora.exponencial,
35         6: calculadora.divisaoresto
36     }
37
38     operacao = ""
39
40     while operacao != 7:
41         print('' [1] - somar
42             [2] - subtrair
43             [3] - multiplicar
44             [4] - dividir
45             [5] - exponencial
46             [6] - Sobre divisão
47             [7] - sair'')
48
49         try:
50             operacao = int(input('Qual a operação desejada?'))
51             if operacao == 7:
52                 print("Fim da operação")
53                 break
54
55             if operacao in operador:
56                 try:
57                     numero1 = float(input("Coloque o primeiro número: "))
58                     numero2 = float(input("Coloque o segundo número: "))
59
60                     operacaoescolhida = operador[operacao]
61                     valorfinal = operacaoescolhida(numero1, numero2)
62                     print(f'0 valor final é {valorfinal}')
63                 except ValueError:
64                     print("Entrada inválida! Por favor, digite apenas n
65                 except ZeroDivisionError as e:
66                     print(f"Erro: {e}")
67             else:
68                 print("Operação não suportada pela calculadora")
69         except ValueError:
70             print("Entrada inválida, digite apenas o NÚMERO da operação

```

test\_py

```

1 from calculadorarevisada import Calculadora
2 import pytest
3 import sys
4
5 c = Calculadora()
6 sys.set_int_max_str_digits(15000)
7
8 @pytest.mark.parametrize("numero1, numero2, valorfinal", [
9     (4, -9, -5),
10    (7, 0, 7),
11    (28, 8372, 8400)
12 ])
13
14 def test_soma(numero1, numero2, valorfinal):
15     assert c.soma(numero1, numero2) == valorfinal
16
17 @pytest.mark.parametrize("numero1, numero2, valorfinal", [
18     (4, 9, -5),
19     (-7, 0, -7),
20     (28, -8372, 8400)
21 ])
22
23 def test_subtracao(numero1, numero2, valorfinal):
24     assert c.subtracao(numero1, numero2) == valorfinal
25
26 @pytest.mark.parametrize("numero1, numero2, valorfinal", [
27     (4, -9, -36),
28     (7, 0, 0),
29     (12, -5, -60),
30     (-28, -8372, 234416)
31 ])

```

```

32
33 def test_multiplicacao(numero1, numero2, valorfinal):
34     assert c.multiplicacao(numero1, numero2) == valorfinal
35
36 @pytest.mark.parametrize("numero1, numero2, valorfinal", [
37     (4, -9, -4/9),
38     (-6, -12, 0.5),
39     (28, 8372, 28/8372)
40 ])
41
42 def test_divisao(numero1, numero2, valorfinal):
43     assert c.divisao(numero1, numero2) == valorfinal
44
45 def test_divisao_zero():
46     with pytest.raises(ZeroDivisionError) as exec_info:
47         c.divisao(7, 0)
48     assert "Não é possível dividir por 0" in str(exec_info)
49
50 @pytest.mark.parametrize("numero1, numero2, valorfinal", [
51     (4, 9, 262144),
52     (-12, 2, 144),
53     (7, 0, 1),
54     (28, 8372, 28*8372)
55 ])
56
57 def test_exponencial(numero1, numero2, valorfinal):
58     assert c.exponencial(numero1, numero2) == valorfinal
59
60 @pytest.mark.parametrize("numero1, numero2, valorfinal", [
61     (4, 9, 4),
62     (-14, 6, -2),
63     (28, 8372, 28)
64 ])
65
66 def test_divisaoresto(numero1, numero2, valorfinal):
67     assert c.divisaoresto(numero1, numero2) == valorfinal
68
69 def test_divisaoresto_zero():
70     with pytest.raises(ZeroDivisionError) as exec_info:
71         c.divisaoresto(7, 0)
72     assert "Não é possível dividir por 0" in str(exec_info)

```

02. Autor do código: Eduardo Neves de Souza

Autor do Review: Gustavo Santos Queiroz

Link do repositório: [repo-fase2-aircompany-compass/Calculadora at sprint-1 · eduardo-souza-1/repo-fase2-aircompany-compass](https://github.com/eduardo-souza-1/repo-fase2-aircompany-compass)

Resumo do review:

-  O código apresenta um ótimo domínio de testes unitários, destacando-se pela clareza na nomenclatura das variáveis (num1, num2, resultado), pelo tratamento correto da exceção de divisão por zero, e pelo bom uso do @pytest.mark.parametrize para validar múltiplos cenários de forma limpa e direta. Como oportunidades de melhoria para o aprimoramento de boas práticas de código, recomendo a utilização de @pytest.fixture nos testes para evitar a repetição da instância da calculadora (calculo = Calculadora()) e a inclusão do parâmetro match no pytest.raises para garantir que a mensagem exata do erro seja validada (with pytest.raises(ValueError, match="Impossível dividir por zero.")). No geral, o código está limpo e objetivo e pode ficar ainda mais ágil. Particularmente, o que consegui avaliar e enxergar melhor foi a agilidade do uso de parametrize, o qual deixei de fora do meu próprio código por achar a tarefa simples demais.

Código refeito pelo autor:

calculadora.py

```

1 class Calculadora:
2
3     def somar(self, num1, num2):
4         return num1 + num2
5
6     def subtrair(self, num1, num2):
7         return num1 - num2
8

```

```

9     def multiplicar(self, num1, num2):
10         return num1 * num2
11
12     def dividir(self, num1, num2):
13         if num2 == 0:
14             raise ValueError("Impossível dividir por zero.")
15         return num1 / num2

```

test\_basico.py

```

1  import pytest
2  from calculadora import Calculadora
3
4  @pytest.fixture
5  def calc():
6      return Calculadora()
7
8  @pytest.mark.parametrize("num1, num2, resultado", [
9      (5, 4, 9),
10     (-6, -6, -12),
11     (5, 0, 5),
12 ])
13 def test_somar(calc, num1, num2, resultado):
14     assert calc.somar(num1, num2) == resultado
15
16 @pytest.mark.parametrize("num1, num2, resultado", [
17     (10, 3, 7),
18     (3, 7, -4),
19     (9, 0, 9),
20 ])
21 def test_subtrair(calc, num1, num2, resultado):
22     assert calc.subtrair(num1, num2) == resultado
23
24 @pytest.mark.parametrize("num1, num2, resultado", [
25     (5, 5, 25),
26     (5, 0, 0),
27     (-3, 4, -12),
28 ])
29 def test_multiplicar(calc, num1, num2, resultado):
30     assert calc.multiplicar(num1, num2) == resultado
31
32 @pytest.mark.parametrize("num1, num2, resultado", [
33     (10, 2, 5),
34     (7, 2, 3.5),
35 ])
36 def test_dividir(calc, num1, num2, resultado):
37     assert calc.dividir(num1, num2) == resultado
38
39 def test_dividir_por_zero(calc):
40     with pytest.raises(ValueError, match="Impossível dividir por zero.")
41         calc.dividir(6, 0)

```

03. Autor do código: Gustavo Santos Queiroz

Autor do Review: Ana Leticia De Araujo

Link do repositório: [GitHub - gsqxz/calculadora\\_pytest: Calculadora com classe e testes básicos feitos em pytest](https://github.com/gsqxz/calculadora_pytest)

Resumo do review:

- 📌 Tanto a calculadora como os testes estão muito bem feitos, apresentando todos os requisitos do desafio. Na calculadora, durante o code review, eu sugiro a adição de tipos de caracteres (ex: a: float), o que impede que o usuário envie strings ou valores inválidos. Já nos testes, o principal seria a utilização do `@pytest.mark.parametrize`, garantindo que sejam realizados diversos testes utilizando somente uma função de teste, facilitando inclusive a adição de novos testes. Além disso, seria interessante a inclusão de valores como expoentes negativos (casos de borda), para assim aumentar a cobertura dos testes. Por fim, seria interessante o uso do `@pytest.raises` para verificação de exceções como o número 0 no divisor em uma operação de divisão.

Código refeito pelo autor:

calculadora.py

```

1  class Calculadora:

```

```

2      # Adição de float para permitir números decimais e impedir strings
3      def soma(self, a: float, b: float) -> float:
4          return a + b
5
6      def subtracao(self, a: float, b: float) -> float:
7          return a - b
8
9      def multiplicacao(self, a: float, b: float) -> float:
10         return a * b
11
12     def divisao(self, a: float, b: float) -> float:
13         if b == 0:
14             raise ValueError("Não é possível dividir por zero")
15         return a / b
16
17     def potenciacao(self, base: float, expoente: float) -> float:
18         return base ** expoente
19
20     def porcentagem(self, valor: float, percentual: float) -> float:
21         return valor * (percentual / 100)

```

test\_calculadora.py

```

1  import pytest
2  from calculadora import Calculadora
3
4  @pytest.fixture
5  def calc():
6      return Calculadora()
7
8  # Adição de parametrize para realizar diversos testes com uma só função
9  @pytest.mark.parametrize("a, b, resultado_esperado", [
10     (3, 3, 6),
11     (-1, 1, 0),
12     (2.5, 2.5, 5.0) # Caso com números decimais sugerido
13 ])
14 def test_soma(calc, a, b, resultado_esperado):
15     assert calc.soma(a, b) == resultado_esperado
16
17
18 @pytest.mark.parametrize("a, b, resultado_esperado", [
19     (10, 5, 5),
20     (0, 5, -5),
21     (5.5, 2.0, 3.5)
22 ])
23 def test_subtracao(calc, a, b, resultado_esperado):
24     assert calc.subtracao(a, b) == resultado_esperado
25
26
27 @pytest.mark.parametrize("a, b, resultado_esperado", [
28     (4, 3, 12),
29     (5, 0, 0),
30     (-2, 3, -6) # Caso com número negativo sugerido
31 ])
32 def test_multiplicacao(calc, a, b, resultado_esperado):
33     assert calc.multiplicacao(a, b) == resultado_esperado
34
35
36 @pytest.mark.parametrize("a, b, resultado_esperado", [
37     (50, 2, 25),
38     (9, 3, 3),
39     (7, 2, 3.5) # Caso com resultado decimal sugerido
40 ])
41 def test_divisao(calc, a, b, resultado_esperado):
42     assert calc.divisao(a, b) == resultado_esperado
43
44
45 def test_divisao_por_zero(calc):
46     with pytest.raises(ValueError, match="Não é possível dividir por zero"):
47         calc.divisao(10, 0)
48
49
50 @pytest.mark.parametrize("base, expoente, resultado_esperado", [
51     (5, 2, 25),
52     (10, 0, 1),
53     (2, -1, 0.5), # Caso de borda sugerido: expoente negativo
54 ])
55 def test_potenciacao(calc, base, expoente, resultado_esperado):
56     assert calc.potenciacao(base, expoente) == resultado_esperado
57

```

```
58
59 @pytest.mark.parametrize("valor, percentual, resultado_esperado", [
60     (200, 15, 30.0),
61     (50, 50, 25.0),
62     (100, 0, 0.0)
63 ])
64 def test_porcentagem(calc, valor, percentual, resultado_esperado):
65     assert calc.porcentagem(valor, percentual) == resultado_esperado
```

---